

Genetic Algorithm for Economic Dispatch

استخدام الخوارزمية الجينية في التوزيع الاقتصادي

Cost Minimization, Power Balance, Constraints, Penalty Functions, and GA

تقليل التكلفة وتوازن القدرة والقيود ودوال العقوبة والخوارزمية الجينية

د. م. أوس غباش | إعداد: Dr.-Ing. Aouss Gabash

Lab Objectives

- Formulate the Economic Dispatch problem mathematically.
- Represent generator powers as a chromosome.
- Define a generation-cost objective function.
- Handle generator limits and power-balance constraints.
- Use a penalty function for infeasible solutions.
- Implement a simplified Genetic Algorithm in MATLAB.
- Track the best cost across generations.

أهداف المختبر

- صياغة مسألة التوزيع الاقتصادي رياضيًا.
- تمثيل استطاعات المولدات على شكل كروموسوم.
- تعريف دالة تكلفة التوليد.
- التعامل مع حدود المولدات وقيد توازن القدرة.
- استخدام دالة عقوبة للحلول غير الممكنة.
- تنفيذ خوارزمية جينية مبسطة في MATLAB.

1. Economic Dispatch Problem

Economic Dispatch determines the generator outputs that satisfy demand at minimum operating cost.

$$\text{Minimize } J = \sum C_i(P_i)$$

subject to:

$$\sum P_i = P_{load}$$

$$P_{i, \min} \leq P_i \leq P_{i, \max}$$

1. مسألة التوزيع الاقتصادي

تهدف مسألة التوزيع الاقتصادي إلى تحديد استطاعات التوليد التي تحقق الطلب الكهربائي بأقل تكلفة تشغيلية.

$$\text{Minimize } J = \sum C_i(P_i)$$

2. Two-Generator System

Consider two thermal generators.

Generator	Pmin [MW]	Pmax [MW]
G1	50	200
G2	50	180

Load demand:

$$P_{load} = 250MW$$

2. نظام مكون من مولدين

لدينا مولدان حراريان بحدود تشغيلية مختلفة.

$$P_{load} = 250MW$$

3. Generator Cost Functions

Assume quadratic fuel-cost functions:

$$C_1(P_1) = a_1 P_1^2 + b_1 P_1 + c_1$$

$$C_2(P_2) = a_2 P_2^2 + b_2 P_2 + c_2$$

Use:

$$C_1 = 0.004 P_1^2 + 5.3 P_1 + 500$$

$$C_2 = 0.006 P_2^2 + 5.5 P_2 + 400$$

3. دوال تكلفة المولدات

نفترض أن تكلفة الوقود لكل مولد تتبع دالة تربيعية.

$$C_i(P_i) = a_i P_i^2 + b_i P_i + c_i$$

هذه صيغة شائعة في النماذج التعليمية لمسائل التوزيع الاقتصادي.

4. Total Operating Cost

$$J(P_1, P_2) = C_1(P_1) + C_2(P_2)$$

The optimization goal is:

$$\min J(P_1, P_2)$$

4. التكلفة التشغيلية الكلية

$$J(P_1, P_2) = C_1(P_1) + C_2(P_2)$$

نبحث عن قيم P_1 و P_2 التي تجعل التكلفة الكلية أصغر ما يمكن.

5. Power-Balance Constraint

Ignoring transmission losses in this educational example:

$$P_1 + P_2 = P_{load}$$

Therefore:

$$P_1 + P_2 = 250MW$$

5. قيد توازن القدرة

نهمل الفواقد في هذا المثال التعليمي، ولذلك يجب أن يكون مجموع التوليد مساويًا تمامًا للحمل.

$$P_1 + P_2 = 250MW$$

6. Chromosome Representation

One GA chromosome represents one candidate dispatch:

$$x = [P_1, P_2]$$

Example:

$$x = [140, 110]$$

This solution satisfies:

$$140 + 110 = 250MW$$

6. تمثيل الكروموسوم

يمثل كل كروموسوم حلاً مرشحاً لمسألة التوزيع الاقتصادي.

$$x = [P_1, P_2]$$

7. Why Do We Need a Penalty?

A randomly generated chromosome may not satisfy power balance.

Example:

$$x = [120, 90]$$

$$P_1 + P_2 = 210 \neq 250$$

We penalize such infeasible solutions.

7. لماذا نحتاج إلى العقوبة؟

قد ينتج المجتمع العشوائي حلولاً لا تحقق توازن القدرة.

الحل غير الممكن يجب أن يصبح أقل جاذبية للخوارزمية.

8. Penalty Function

Define the power mismatch:

$$\Delta P = P_1 + P_2 - P_{load}$$

Then define a penalized objective:

$$J_{pen} = J + \lambda(\Delta P)^2$$

where λ is a large penalty coefficient.

8. دالة العقوبة

نحسب أولاً عدم توازن القدرة:

$$\Delta P = P_1 + P_2 - P_{load}$$

ثم نضيف عقوبة مربعة إلى دالة الهدف:

$$J_{pen} = J + \lambda(\Delta P)^2$$

كلما ازداد انتهاك قيد التوازن، ارتفعت العقوبة بسرعة.

9. Fitness Function

For a minimization problem, one possible fitness is:

$$F = 1/(1 + J_{pen})$$

Lower penalized cost → higher fitness.

9. دالة الملاءمة

$$F = 1/(1 + J_{pen})$$

الحلول ذات التكلفة الأقل تحصل على ملاءمة أكبر.

10. GA Parameters

```
popSize = 40;  
maxGen  = 100;  
  
Pc = 0.8;  
Pm = 0.1;  
  
lambda = 1000;
```

where:

- Pc = crossover probability
- Pm = mutation probability
- lambda = penalty coefficient

10. معاملات الخوارزمية الجينية

نحدد حجم المجتمع وعدد الأجيال واحتمال التزاوج واحتمال الطفرة ومعامل العقوبة.

11. Generate Initial Population

```

P1min = 50;
P1max = 200;

P2min = 50;
P2max = 180;

population = zeros(popSize,2);

population(:,1) = ...
    P1min + rand(popSize,1)*(P1max-P1min);

population(:,2) = ...
    P2min + rand(popSize,1)*(P2max-P2min);

```

11. توليد المجتمع الابتدائي

يتم توليد قيم P_1 و P_2 عشوائيًا ضمن الحدود المسموحة لكل مولد.

12. Evaluate One Candidate

For chromosome:

$$x = [P_1, P_2]$$

calculate:

$$C_1 = 0.004P_1^2 + 5.3P_1 + 500$$

$$C_2 = 0.006P_2^2 + 5.5P_2 + 400$$

$$J = C_1 + C_2$$

$$J_{pen} = J + \lambda(P_1 + P_2 - P_{load})^2$$

12. تقييم حل واحد

لكل كروموسوم نحسب تكلفة المولدين، ثم التكلفة الكلية، ثم عقوبة عدم توازن القدرة.

13. MATLAB Cost Evaluation

```
P1 = population(i,1);
P2 = population(i,2);

C1 = 0.004*P1^2 + 5.3*P1 + 500;
C2 = 0.006*P2^2 + 5.5*P2 + 400;

cost = C1 + C2;

mismatch = P1 + P2 - Pload;

penCost = cost + lambda*mismatch^2;
```

13. حساب التكلفة في MATLAB

تنفذ هذه الأسطر دالة الهدف والعقوبة مباشرة.

14. Tournament Selection

A simple selection method is tournament selection.

Choose two random candidates and keep the one with lower penalized cost.

```
i1 = randi(popSize);  
i2 = randi(popSize);  
  
if penCostVec(i1) < penCostVec(i2)  
    parent = population(i1,:);  
else  
    parent = population(i2,:);  
end
```

14. اختيار البطولة

نختار حلين عشوائيًا، ثم نحتفظ بالحل الذي يمتلك تكلفة معاقبة أقل.

15. Arithmetic Crossover

For parents p_1 and p_2 :

$$child_1 = \alpha p_1 + (1 - \alpha)p_2$$

$$child_2 = (1 - \alpha)p_1 + \alpha p_2$$

```
alpha = rand;

child1 = alpha*parent1 + ...
        (1-alpha)*parent2;

child2 = (1-alpha)*parent1 + ...
        alpha*parent2;
```

15. التزاوج الحسابي

يتم دمج قيم الآباء خطيًا لإنتاج طفلين جديدين.

16. Mutation

A simple Gaussian mutation can be used:

$$P_{new} = P + \sigma r$$

where r is a random normal variable.

```

sigmaMut = 10;

if rand < Pm
    child1(1) = child1(1) + sigmaMut*randn;
end

if rand < Pm
    child1(2) = child1(2) + sigmaMut*randn;
end

```

16. الطفرة

تضيف الطفرة تغييرًا عشوائيًا صغيرًا إلى استطاعة أحد المولدات.

تساعد الطفرة على استكشاف مناطق جديدة من فضاء الحلول.

17. Enforce Generator Limits

After crossover and mutation, clip the offspring:

```

child1(1) = min(max(child1(1),P1min),P1max);
child1(2) = min(max(child1(2),P2min),P2max);

child2(1) = min(max(child2(1),P1min),P1max);
child2(2) = min(max(child2(2),P2min),P2max);

```

17. فرض حدود المولدات

بعد التزاوج والطفرة نتأكد أن استطاعات التوليد ما تزال ضمن حدودها المسموحة.

18. Elitism

Keep the best solution from the current generation:

```
[bestCost,bestIdx] = min(penCostVec);  
  
elite = population(bestIdx,:);
```

The best candidate should not be lost during random evolutionary operations.

18. النخبوية

يتم الاحتفاظ بأفضل حل في كل جيل ونقله مباشرة إلى الجيل التالي.

19. Complete GA Logic



19. منطق الخوارزمية الكامل

مجتمع ابتدائي ← تقييم التكلفة ← الاختيار ← التزاوج ← الطفرة ← الجيل الجديد ← التكرار

20. Complete MATLAB Program

```

clear; clc; close all;

% -----
% Genetic Algorithm for Economic Dispatch
% Two-generator educational example
% -----

Pload = 250;

% Generator limits
P1min = 50;
P1max = 200;

P2min = 50;
P2max = 180;

% GA parameters
popSize = 40;
maxGen = 100;

Pc = 0.8;
Pm = 0.1;

sigmaMut = 10;

lambda = 1000;

% -----
% Initial population
% -----

population = zeros(popSize,2);

population(:,1) = ...
    P1min + rand(popSize,1)*(P1max-P1min);

population(:,2) = ...
    P2min + rand(popSize,1)*(P2max-P2min);

```

```

bestHist = zeros(maxGen,1);

% -----
% GA loop
% -----

for gen = 1:maxGen

    costVec      = zeros(popSize,1);
    penCostVec = zeros(popSize,1);

    % -----
    % Evaluate population
    % -----

    for i = 1:popSize

        P1 = population(i,1);
        P2 = population(i,2);

        C1 = 0.004*P1^2 + 5.3*P1 + 500;
        C2 = 0.006*P2^2 + 5.5*P2 + 400;

        cost = C1 + C2;

        mismatch = P1 + P2 - Pload;

        penCost = ...
            cost + lambda*mismatch^2;

        costVec(i) = cost;
        penCostVec(i) = penCost;

    end

    % -----
    % Best solution
    % -----

    [bestPenCost,bestIdx] = min(penCostVec);

```

```

elite = population(bestIdx,:);

bestHist(gen) = bestPenCost;

% -----
% Create new population
% -----

newPopulation = zeros(size(population));

newPopulation(1,:) = elite;

k = 2;

while k <= popSize

    % -----
    % Tournament selection for Parent 1
    % -----

    i1 = randi(popSize);
    i2 = randi(popSize);

    if penCostVec(i1) < penCostVec(i2)
        parent1 = population(i1,:);
    else
        parent1 = population(i2,:);
    end

    % -----
    % Tournament selection for Parent 2
    % -----

    i1 = randi(popSize);
    i2 = randi(popSize);

    if penCostVec(i1) < penCostVec(i2)
        parent2 = population(i1,:);
    else

```

```

        parent2 = population(i2,:);
    end

    % -----
    % Crossover
    % -----

    if rand < Pc

        alpha = rand;

        child1 = ...
            alpha*parent1 + ...
            (1-alpha)*parent2;

        child2 = ...
            (1-alpha)*parent1 + ...
            alpha*parent2;

    else

        child1 = parent1;
        child2 = parent2;

    end

    % -----
    % Mutation
    % -----

    for j = 1:2

        if rand < Pm
            child1(j) = ...
                child1(j) + sigmaMut*randn;
        end

        if rand < Pm
            child2(j) = ...
                child2(j) + sigmaMut*randn;
        end
    end

```

```

        end

    end

    % -----
    % Generator limits
    % -----

    child1(1) = ...
        min(max(child1(1),P1min),P1max);

    child1(2) = ...
        min(max(child1(2),P2min),P2max);

    child2(1) = ...
        min(max(child2(1),P1min),P1max);

    child2(2) = ...
        min(max(child2(2),P2min),P2max);

    % -----
    % Store offspring
    % -----

    newPopulation(k,:) = child1;

    if k+1 <= popSize
        newPopulation(k+1,:) = child2;
    end

    k = k + 2;

end

population = newPopulation;

end

% -----
% Final evaluation

```

```

% -----

finalPenCost = zeros(popSize,1);
finalCost     = zeros(popSize,1);

for i = 1:popSize

    P1 = population(i,1);
    P2 = population(i,2);

    C1 = 0.004*P1^2 + 5.3*P1 + 500;
    C2 = 0.006*P2^2 + 5.5*P2 + 400;

    finalCost(i) = C1 + C2;

    mismatch = P1 + P2 - Pload;

    finalPenCost(i) = ...
        finalCost(i) + lambda*mismatch^2;

end

[~,idx] = min(finalPenCost);

Pbest = population(idx,:);

P1best = Pbest(1);
P2best = Pbest(2);

C1best = ...
    0.004*P1best^2 + 5.3*P1best + 500;

C2best = ...
    0.006*P2best^2 + 5.5*P2best + 400;

totalCost = C1best + C2best;

mismatch = P1best + P2best - Pload;

% -----

```

```

% Display results
% -----

fprintf('Best Economic Dispatch Solution\n\n')

fprintf('P1 = %.4f MW\n',P1best)
fprintf('P2 = %.4f MW\n',P2best)

fprintf('Total Generation = %.4f MW\n', ...
        P1best+P2best)

fprintf('Load Demand = %.4f MW\n',Pload)

fprintf('Power Mismatch = %.6f MW\n\n', ...
        mismatch)

fprintf('Generation Cost = %.4f\n', ...
        totalCost)

% -----
% Convergence curve
% -----

figure

plot(1:maxGen,bestHist,'LineWidth',1.6)

grid on

xlabel('Generation')
ylabel('Best Penalized Cost')

title('GA Convergence for Economic Dispatch')

```

20. البرنامج الكامل في MATLAB

ينفذ البرنامج مجتمعًا أوليًا، ويحسب التكلفة والعقوبة، ثم يكرر عمليات الاختيار والتزاوج والطفرة والنخبة حتى الوصول إلى حل اقتصادي جيد.

21. Understanding the Convergence Curve

The graph:

Generation $\rightarrow$ Best Penalized Cost

should generally decrease as better solutions are discovered.

Because GA is stochastic, the curve and final result may differ slightly between independent runs.

21. فهم منحنى التقارب

يجب أن تنخفض أفضل تكلفة مع تقدم الأجيال بصورة عامة.

الخوارزمية الجينية عشوائية جزئيًا، ولذلك قد تختلف النتائج قليلًا بين تشغيل وآخر.

22. Why the Penalty Coefficient Matters

If λ is too small:

The algorithm may prefer a cheap solution that does not satisfy the load.

If λ is very large:

Constraint satisfaction becomes dominant, but the numerical search can become harder.

22. أهمية معامل العقوبة λ

إذا كانت λ صغيرة جدًا، فقد تقبل الخوارزمية حلولاً رخيصة لكنها لا تحقق توازن القدرة. أما إذا كانت كبيرة جدًا، تصبح العقوبة مهيمنة بصورة قوية على دالة الهدف.

23. Experiment 1 — Change the Load

Try:

```
Pload = 200;  
Pload = 250;  
Pload = 300;
```

Compare generator outputs and total generation cost.

23. التجربة 1 — تغيير الحمل

غيّر الحمل وشاهد كيف تتغير استطاعات المولدات والتكلفة الاقتصادية الكلية.

24. Experiment 2 — Change Population Size

```
popSize = 10;  
popSize = 40;  
popSize = 100;
```

Larger populations improve exploration but require more objective-function evaluations.

24. التجربة 2 — تغيير حجم المجتمع

قارن جودة الحل وزمن الحساب عند استخدام أحجام مختلفة للمجتمع.

25. Experiment 3 — Change Mutation Probability

```
Pm = 0.01;  
Pm = 0.10;  
Pm = 0.40;
```

Observe convergence and variability between runs.

25. التجربة 3 — تغيير احتمال الطفرة

إذا كانت الطفرة منخفضة جدًا قد يقل التنوع، وإذا كانت عالية جدًا قد تصبح عملية البحث عشوائية أكثر من اللازم.

26. Engineering Improvement

For this two-generator problem, power balance can also be enforced directly:

$$P_2 = P_{load} - P_1$$

Then only P_1 needs to be optimized.

This is an important engineering idea: use problem structure whenever possible, instead of asking the optimizer to discover every constraint numerically.

26. تحسين هندسي للمسألة

يمكن فرض توازن القدرة مباشرة:

$$P_2 = P_{load} - P_1$$

وعندها يكفي البحث عن متغير قرار واحد فقط.

كلما استخدمنا المعرفة الفيزيائية للمسألة، أمكن تبسيط عملية التحسين.

27. Extension to Power Losses

A more realistic Economic Dispatch includes transmission losses:

$$\sum P_i = P_{load} + P_{loss}$$

Then the chromosome and objective evaluation must also account for system losses.

27. إضافة فواقد الشبكة

في الحالة الأكثر واقعية يصبح توازن القدرة:

$$\sum P_i = P_{load} + P_{loss}$$

وهذا يربط المسألة بصورة أكبر بتحليل تدفق القدرة.

28. From Economic Dispatch to OPF

Economic Dispatch mainly focuses on active-power generation and cost.

Optimal Power Flow adds network variables and constraints:

$$x = [PG, QG, V, \theta, Tap, \dots]$$

with constraints such as:

PowerFlowEquations

$$V_{min} \leq V \leq V_{max}$$

$$I \leq I_{max}$$

This is the natural path from this laboratory toward AI-based OPF.

28. من التوزيع الاقتصادي إلى تدفق القدرة الأمثل

يضيف OPF إلى تكلفة التوليد معادلات الشبكة والجهود والزوايا وحدود الخطوط والقدرة الردية وغيرها من القيود.

وهذه هي الخطوة الطبيعية نحو تطبيقات الذكاء الاصطناعي والتحسين في OPF.

29. Lab Tasks

1. Define generator cost functions.
2. Generate the initial population.
3. Calculate power mismatch for each chromosome.
4. Implement the penalty function.
5. Implement tournament selection.

6. Implement arithmetic crossover.
7. Implement mutation and generator limits.
8. Plot the convergence curve.
9. Record the final P_1 and P_2 .
10. Verify power balance manually.
11. Change load demand and repeat the experiment.
12. Change population and mutation parameters.
13. Explain how this problem can be extended to OPF.

29. مهام المختبر

1. عرّف دوال تكلفة المولدات.
2. أنشئ المجتمع الابتدائي.
3. احسب عدم توازن القدرة لكل كروموسوم.
4. نفذ دالة العقوبة.
5. نفذ اختيار البطولة.
6. نفذ التزاوج الحسابي.
7. نفذ الطفرة وحدود المولدات.
8. ارسم منحنى التقارب.
9. سجل القيم النهائية لـ P_1 و P_2 .
10. تحقق يدويًا من توازن القدرة.
11. غيّر الحمل وأعد التجربة.
12. غيّر حجم المجتمع واحتمال الطفرة.
13. اشرح كيف يمكن تطوير المسألة إلى OPF.



Review Questions

1. What is the objective of Economic Dispatch?
2. What does one chromosome represent?
3. Why is a penalty function required?
4. What happens if λ is too small?

5. What is tournament selection?
6. Why do we use mutation?
7. Why is elitism useful?
8. How do generator limits affect the search?
9. How can power balance be enforced directly?
10. What additional constraints appear in OPF?

أسئلة المراجعة

1. ما الهدف من التوزيع الاقتصادي؟
2. ماذا يمثل الكروموسوم الواحد؟
3. لماذا نحتاج إلى دالة عقوبة؟
4. ماذا يحدث إذا كانت λ صغيرة جدًا؟
5. ما المقصود باختيار البطولة؟
6. لماذا نستخدم الطفرة؟
7. ما فائدة النخبوية؟
8. كيف تؤثر حدود المولدات على البحث؟
9. كيف يمكن فرض توازن القدرة مباشرة؟
10. ما القيود الإضافية التي تظهر في OPF؟